

Engineering Log — What we tried before Recipe v3 worked

Failures, audits, bugs, and the discipline that made the headline publishable

Author: Aryan Banwala · DTU CSE · B.Tech-II

Generated: 2026-05-18

This is the engineering slice of the project. Not the headline numbers — the work that made the headline numbers *defensible*. Three major experiments that did not become the headline (Recipe v2 intermediate; Intervention 2 negative; hubness post-processing negative), the Phase 1 audit that caught four real latent bugs, three engineering gotchas that ate hours and are now permanent memory notes, and the methodological choices (pre-registration, multiple controls, cross-corpus tuning) that anchor every claim.

Each item is sized in proportion to how much intellectual work it represented. Major experiments get full anatomy (Hypothesis / Design / Result / Lesson). Small fixes get a paragraph. Every claim cites a file path in the repo. No hallucinations.

Companion docs. Deep math: `AFP_Teaching_Guide.pdf`. High-level overview: `AFP_Overview_Guide.pdf`.

Reproducibility. Every claim is grounded in a file in the repo (`data/results/...`) or a peer-reviewed paper. Pre-registration protocols are committed in git. No hallucinations.

Artifacts.

Live demo · `huggingface.co/spaces/Tachyeon/afp-indian-classical-demo`

Dataset · `huggingface.co/datasets/Tachyeon/audio-fingerprint-indian-bench`

Code · `github.com/ipritamdash/afp-indian-classical`

Engineering Log

What we tried before Recipe v3 worked

A senior-researcher's notebook of the experiments, audits, bugs, and engineering discoveries that shaped the final result. Two of these experiments were pre-registered negatives (intentionally published as failures). One was an intermediate single-seed attempt that didn't clear the bar. One was a four-reviewer audit that caught real latent bugs. The rest are engineering gotchas worth documenting.

Each major item below has the same anatomy: **Hypothesis**, **Design** (including controls), **Result**, **Lesson**. Smaller items get a single paragraph. Items are sized in rough proportion to how much of the project's intellectual work they represented.

Every claim in this document is grounded in a file in the repo. File paths are cited inline.

Reading guide

- **Part 1** — three major experiments that *didn't* become the headline (recipe v2 intermediate; Intervention 2 negative; hubness post-processing negative).
- **Part 2** — the Phase 1 audit: four parallel reviewer agents caught four real latent bugs before any of them touched a published number.
- **Part 3** — engineering gotchas: bugs that ate hours and now have permanent fixes documented as memory notes.
- **Part 4** — methodological choices that anchored everything (pre-registration discipline, cross-corpus tuning, multi-control design).

If you're triaging by time: read Part 1 in full (it's the bulk of the work that didn't show up in the headline). Skim Parts 2-4.

Part 1 — Major experiments

Three substantial experiments that produced negative or borderline results. Each one took serious design effort, was pre-registered, and shaped what came next.

1.1 Recipe v2 — the first training-recipe attempt

The first try at fixing NAFP through training. Single seed, 10 epochs, batch size 120. Did not become the headline; motivated the 3-seed × 30-epoch escalation that became Recipe v3.

Hypothesis (pre-registered before training)

Source: `data/results/nafp/recipe_v2_10ep/PROTOCOL.md` (locked 2026-05-14, *before* any training run).

"Applying two of NMFP's (Araz et al. ISMIR 2025) published recipe fixes to NAFP training at the same 10-epoch budget as our existing baseline will reduce the same-artist intra-class confusion failure mode on Saraga 1.5 main_1s queries."

Two specific deltas vs the NAFP baseline, both inherited from NMFP:

Knob	Baseline	Recipe v2	Why
F_MIN	300 Hz	160 Hz	Captures 160-300 Hz band where tanpura drone, vocal fundamentals, tabla low harmonics live
TR_SEG_MODE	all	random_oneshot (re-sampled per epoch)	Removes in-batch false negatives that arise when multiple segments from the same source track land in one mini-batch

Everything else preserved: NT-Xent $\tau=0.05$, BSZ=120, architecture, augmentation pool, training data (FMA-medium 10k_icassp), optimizer (Adam LR=1e-4, cosine schedule), seed=42.

Design

- **Single seed:** `random_seed=42`. Recipe v2's pre-registration explicitly noted "no multi-seed factorial" — that was reserved for follow-up if the single-seed result passed.
- **Primary endpoint:** HR@1 on Saraga main_1s.
- **Decision rule:** success if both `HR@1 > 0.983` AND McNemar exact two-sided binomial test gives `p < 0.05` (unadjusted; 8-cell Bonferroni was NOT pre-registered for v2's primary).
- **Secondary endpoints:** 7 other cells reported descriptively only.
- **Compute:** ≤ 90 minutes on Colab L4. Single training run.

Result

Source: `data/results/nafp/recipe_v2_10ep/main_1s/scores.json`.

```
hr@1 = 0.991 (991 hits / 1 000 queries)
```

Versus baseline `hr@1 = 0.983` (983 hits) — a **+8-hit improvement** (17 misses → 9 misses on the headline cell).

Promising — but the directory does **not contain** a `RESULTS.md` or a McNemar CSV. That's the tell: the recipe v2 result was not promoted to the headline. Single-seed power on a small discordant-pair count (around 8 wins, a few losses) is borderline against an unadjusted McNemar bar, and the project was committed to making the headline Bonferroni-defensible.

Lesson

The recipe works directionally — but at a single seed $\times$ 10 epochs you're operating at the floor of statistical power. To make a defensible claim you need either (a) a much larger discordant-pair count (more epochs, bigger batch → more aggressive recipe-induced changes) or (b) more seeds (pool across them in McNemar).

The path forward was **both**: scale to BSZ=320, 30 epochs, and run **3 seeds**. That's Recipe v3. Recipe v2 isn't a failure — it's the pilot study that justified the compute spend on v3.

1.2 Intervention 2 — per-artist mean subtraction (pre-registered NEGATIVE)

The deepest experiment in this project. Targeted the same-artist failure mode at **inference time** — no retraining. Pre-registered with explicit decision rules and three controls. Result: cleanly **falsified the proposed mechanism**.

Hypothesis (pre-registered before any sweep)

Source: `data/results/nafp/intervention2/PROTOCOL.md` (locked 2026-05-12 *before* any α -sweep was run).

The empirical setup: of the 17 NAFP-baseline misses on `main_1s`, **14 (82 %) were same-artist top-1 confusions** — the system picked a different recording by the same artist as the truth (verified against `data/results/nafp/saraga_only_main_1s/query_results.parquet`).

"There is a per-artist 'hub direction' in NAFP embedding space — a component shared by all segments of a single artist regardless of raaga / work. Subtracting this hub partially from ref embeddings de-confounds artist identity from segment identity and reduces same-artist top-1 confusions."

Design (V1 — Asymmetric per-artist mean subtraction)

For each ref segment `e_seg` belonging to ref `r` with artist `a`:

$$e_seg' = (e_seg - \alpha \cdot \mu_a^{\{LOO(r)\}}) / \|e_seg - \alpha \cdot \mu_a^{\{LOO(r)\}}\|$$

where $\mu_a^{\{LOO(r)\}}$ is the **leave-one-track-out** mean — average of all segments of all refs in artist `a`, *excluding ref r's own segments*. Standard AS-Norm protocol (Matejka 2017).

- **Query side untouched** (no artist label available at test time)
- **Asymmetric rescore** — accepted; the test-time query has no knowable artist
- Pre-computed `centroids_per_ref.npy` shape `(357, 128)` — one LOO centroid per ref

Hyperparameter sweep + controls

Knob	Values
α -sweep	{0.00, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30}
Headline α	0.10 (literature-pinned, NOT data-derived — Kenny 2010, Matejka 2017)
$\alpha = 0$ (built-in sanity)	Must reproduce baseline HR@1 bit-identically; failure aborts the sweep

Three controls at the cross-corpus winner α^* — each falsifies a different "what if our mechanism is wrong"

story:

Control	What it tests
Shuffled-centroid	Assign each ref a centroid from a <i>randomly-chosen other artist</i> (seed=20260512). If V1's gain is "real artist hub", shuffled should not help.
Isotropic-centroid	Replace each centroid with a unit-Gaussian random vector. Tests whether ANY centroid subtraction helps via generic anti-hub effect (Mu & Viswanath 2018) vs whether the artist-specific structure is essential.
$\alpha = 0$ sanity	Identity rescore — must equal baseline bit-for-bit.

Cross-corpus tuning (no peeking): pick α^* on Hindustani-only, evaluate on Carnatic-only, and symmetric. Prevents "the best α on the test set" cherry-picking.

Statistical method

- **McNemar's exact test** (Dietterich 1998) on paired hit/miss outcomes per query
- **Bonferroni correction** across 8 cells $\rightarrow \alpha_{\text{family}} = 0.05 / 8 = \mathbf{0.00625}$
- **Asymmetric regression policy**: gains must clear Bonferroni; regressions must be non-significant

Decision rule (pre-registered)

V1 is **adopted** iff:

1. ≥ 4 of 4 primary main cells show $\Delta\text{HR}@1 \geq 0$
2. ≥ 2 of 4 primary main cells clear McNemar Bonferroni
3. No primary cell shows a Bonferroni-significant regression
4. Shuffled-centroid control shows *substantially smaller* (or negative) $\Delta\text{HR}@1$ than V1

Failing rule 1 or 3 $\rightarrow$ report as negative result, publish honestly, pivot to AS-Norm V2 fallback. Failing rule 4 $\rightarrow$ mechanism is **not** the claimed one; the gain (if any) is generic centroid effect, not artist-specific.

Result

Source: `data/results/nafp/intervention2/analysis/SUMMARY.md` and `results.csv`.

The per-cell headline ($\alpha = 0.10$):

Cell	Baseline	V1 ($\alpha=0.10$)	Δhits	Shuffled Δ	Isotropic Δ
main_1s	0.983	0.983	+0	+0	+2
main_3s	0.998	0.998	+0	+1	+1
main_5s	0.999	0.999	+0	+1	+1
main_10s	1.000	1.000	+0	0	0
ablation_1s	0.979	0.978	-1	-1	0
ablation_3s/5s/10s	1.000	1.000	0	0	0

Pre-registered decision-rule outcomes:

#	Rule	Result
1	$\geq 4/4$ main cells with $\Delta\text{HR} \geq 0$	4/4 ✓
2	$\geq 2/4$ main cells Bonferroni-significant	0/4 ✗
3	No Bonferroni-significant regression	0 regressions ✓
4	Shuffled $< V1$ (mechanism check)	isotropic $\geq V1$ in 4/4 main cells ✗

Verdict (from SUMMARY.md, verbatim)

"Adoption criterion (rules 1–3): FAIL. Mechanism check (rule 4): isotropic $\geq V1$ in 4/4 main cells → mechanism is NOT per-artist; generic anti-hub effect."

Lesson

The isotropic-centroid control is the critical finding. If a random unit vector replacing the learned centroid produces a result **at least as good** as the learned per-artist centroid, then the artist-specific structure was never doing the work. Any benefit comes from the generic "anti-hub" effect documented by Mu & Viswanath (2018) — subtracting *any* mean-ish direction reduces hub bias.

In plain language: same-artist confusion isn't fixable by **embedding-geometry surgery at inference**. The encoder doesn't have a clean "artist hub direction" to remove. The failure is encoder-level.

This finding gets cited in the paper limitations and the recipe v3 motivation. It scopes the next move to training-recipe changes.

1.3 Hubness post-processing (Inverted Softmax + CSLS) — pre-registered NEGATIVE

A second, independent attempt at fixing same-artist confusion **without retraining**. Tested two well-known anti-hubness rescoreing methods from cross-lingual retrieval. Result: zero effect on any cell. Confirmed Intervention 2's finding.

Hypothesis

NAFP's same-artist failure mode is a generic *hubness* problem — some ref vectors are everyone's nearest neighbour, biasing retrieval. Anti-hubness rescoreing should reduce this bias.

Two methods tested side-by-side (both standard in cross-lingual word-vector retrieval):

- **Inverted Softmax** (Smith et al. 2017): rescore each candidate by subtracting a soft-normalized term derived from how popular it is among other queries.
- **CSLS** (Cross-domain Similarity Local Scaling, Conneau et al. 2018): rescore by subtracting half-sums of each side's k-NN similarity averages.

Design

- 8 cells: Saraga main + ablation × {1s, 3s, 5s, 10s}
- Anchor pool: 4 096 random queries (seed=20260513) for computing the anti-hubness terms
- Compare baseline / inv_softmax / csls side-by-side on the same query results

Result

Source: `data/results/nafp/hubness_postproc/main_1s/scores.json`.

```
baseline:      hr@1 = 0.983   (983/1000)
inv_softmax:   hr@1 = 0.983   (983/1000)  Δ = 0.000
csls:          hr@1 = 0.983   (983/1000)  Δ = 0.000
```

Same numbers across all 8 cells. No gain. No regression. Zero delta on every cell.

Verdict

REJECTED. Both methods reproduced baseline HR@1 *exactly* across the entire 8-cell benchmark.

Lesson

Together with Intervention 2, this rules out inference-time fixes entirely. The remaining lever is training-recipe modifications — which is exactly the direction Recipe v2 / Recipe v3 took.

Two pre-registered negatives, with falsification criteria committed before measurement, both confirming the encoder needs to change, not the inference path. That's the methodologically clean way to scope a follow-up.

Part 2 — The Phase 1 audit

A senior engineer doesn't trust their own pipeline without a second pair of eyes. We organized a multi-agent review of v0.4 — author's 5-layer audit + four independent reviewer agents in parallel. The audit caught four real bugs and forced a re-release.

Source: `docs/post_mortem_2026-05-12.md` (full 294-line audit report).

How it was organized

- **Author's 5-layer audit** ran first: reference library integrity, test set determinism, NAFP output sanity, scoring-code unit tests, cross-system comparability. Covered 30+ specific checks, every one re-runnable from `uv run python` in the repo.
- **Four reviewer agents in parallel**, each with a narrow brief:
 - Reviewer A — test-set adversarial (search for query/ref pairings that shouldn't work)
 - Reviewer B — NAFP output forensic (look at the embedding manifold geometry)
 - Reviewer C — scoring-code review (the `scripts/score.py` module specifically)
 - Reviewer D — cross-system comparability (whether HR@1 is even directly comparable)

across the four systems)

The four real bugs caught (BLOCKER / MAJOR)

Finding	Severity	What it was
B1 (Reviewer A, F18)	BLOCKER	<code>has_twin_in_library</code> was computed from works-text matches, not work-MBIDs. Generic form-names (Thillana, Ragam Thanam Pallavi, Dekho Ajab Khel Hein) over-flagged. 35 of the 165 flagged queries were spurious . Independent author verification confirmed exactly 35/165.
B2 (Reviewer D)	BLOCKER	<code>dejavu_runner.py:397</code> hardcoded <code>ref_stop = ref_start + 10.0</code> . All 1/3/5 s dejavu splits silently emitted 10 s spans regardless of query length. Verified empirically by parquet diff — all 4 splits had <code>ref_stop - ref_start = 10.0</code> . Cosmetic effect on HR/MRR but the column was actively wrong.
B3 (Reviewer C, BUG-2)	MAJOR	<code>score.py:133</code> used a results-derived denominator for <code>no_match</code> . A query that the system failed to emit any row for was silently dropped from <code>frac_no_match</code> . Verified on a crafted 5-query test: reported <code>frac_no_match = 0.2</code> instead of the correct <code>0.8</code> .
M9 (Reviewer D)	MAJOR (latent)	NAFP used <code>np.argsort(...)</code> without <code>kind='stable'</code> . Non-deterministic on cosine ties. 0 actual ties on current data — latent only — but the fix is one line and was applied proactively.

Independent author verification rejected one false-positive agent claim (Reviewer B briefly thought all 8 NAFP result dirs were Mac-encoded — they weren't). Documenting the rejection in the post-mortem is itself part of the methodology.

Other findings (MAJOR, no-fix-required disclosures)

Tag	Disclosure
M3	NAFP <code>align_err_median = 0.125 s</code> on <code>main_1s</code> is a 0.5 s hop quantization artefact , not algorithmic deficiency. <code>top1_near_at_0.05s</code> is structurally unfair to NAFP.
M4	61 / 632 ablation queries time-overlap with main queries on the same ref. Main vs ablation are not statistically independent.
M5	tani N = 11, Carnatic-only — per-bucket tani comparisons impossible.
M6	<code>match_count</code> column semantically incompatible across systems (Olaf hash count, Dejavu hash count at 10× scale, Panako fingerprint count, NAFP constant L). Parquet column only; <code>score.py</code> unaffected.
M7	Latency timers wrap different boundaries per runner — Panako excludes JVM startup + ffmpeg decode entirely. Cross-system latency table needs a footnote.
M8	HR@5 / HR@10 not directly comparable. Olaf emits 1-10 rows per query, Dejavu emits 10, Panako 1-3, NAFP 10. K-asymmetry inflates HR@K>1 for Dejavu and NAFP. HR@1 unaffected.

Fixes applied (v0.5 release)

Bug	Fix
-----	-----

B1	Recompute <code>has_twin_in_library</code> using <code>work_mbid</code> equality only. Net change: 165 spurious → 130 genuine twins .
B2	<code>dejavu_runner.py:397</code> now reads <code>length_sec</code> from the query manifest.
B3	<code>score.py:133</code> uses set-difference against the manifest.
M9	<code>nafp_runner.py:266</code> uses <code>np.argsort(..., kind='stable')</code> .

Headline HR@1 numbers unchanged. Only the twin/no-twin breakdown moved (Panako with-twin shifted from 165 to 130 denominator, making the genuine twin failure rate more visible — more honest, not less).

Lesson

A 5-layer author audit plus four parallel reviewer agents caught **four BLOCKER/MAJOR** bugs that would have shipped silently. Three were genuine data-correctness issues, one was a latent determinism bug. The audit also surfaced six MAJOR disclosures that became Limitations section material.

The post-mortem document itself is part of the deliverable. Anyone reviewing the project can `git log -- docs/post_mortem_2026-05-12.md` and verify the date predates the v0.5 release.

Part 3 — Engineering discoveries

Three engineering gotchas that consumed hours, now permanently documented as memory notes so they can't bite a future engineer the same way.

3.1 `TF_USE_LEGACY_KERAS=1` silent partial-restore

Discovered during Intervention 2 debugging. TensorFlow 2.19 silently partial-restored the NAFP checkpoint when loaded under the default Keras 3 API: the CNN backbone weights loaded correctly, but the DivEnc head failed silently. The model returned **random 128-D vectors** with **no error raised** — the demo "ran" but every query produced garbage.

Fix: set `os.environ["TF_USE_LEGACY_KERAS"] = "1"` **before any TensorFlow import** in every script that loads NAFP weights. Permanent fix in `demo/app.py` line 19. Documented as a memory note (`feedback_nafp_mac_inference_gotcha.md`) so any future inference script gets it for free.

3.2 Kaggle phone-verify silent CPU downgrade

Kaggle's policy: free GPU access requires a phone-verified account. Unverified accounts get **silently downgraded to CPU** without any error or warning. A training run that should have taken 45 minutes was on track for ~75 hours.

Fix: a fail-fast 2048×2048 matrix multiply at the start of every training kernel. Healthy T4 runs it in < 30 ms; CPU takes > 500 ms. The assertion fires before any compute is wasted. Documented as `feedback_kaggle_phone_verify.md` so the assertion is mandatory in any new training script.

3.3 The Hugging Face Spaces 3-failure build cascade

The live demo's Space build failed three times in sequence before working, each failure revealing a different version pinning issue:

Failure	Cause	Fix
<code>tensorflow==2.19.0</code> <code>not found</code>	HF Spaces default Python is 3.13; TF 2.19 has no Py 3.13 wheels	Pin <code>python_version: "3.11"</code> in the README YAML frontmatter
numpy version conflict	TF 2.19 requires <code>numpy<2.2, >=1.26</code> ; kapre 0.3.7 PyPI rebuild requires <code>numpy>=2.0</code>	Pin intersection: <code>numpy>=2.0, <2.2</code>
librosa version conflict	kapre 0.3.7 PyPI rebuild requires <code>librosa>=0.11</code> ; default resolver picked 0.10	Pin <code>librosa>=0.11, <1.0</code>

The full final pinned `requirements.txt` is now non-negotiable for any redeployment. Documented in `demo/DEPLOY.md`.

3.4 Twin-flag text matching → MBID-only

Already covered in Part 2 (B1) but worth re-stating standalone: the leakage / twin detector originally used **textual work-name matching**, which over-flagged 35 of 165 main queries because of generic Indian classical form-names. The fix was a one-line change to compare canonical MusicBrainz work-MBIDs instead. Result: **165 → 130 confirmed twins**. The headline tables now report HR@1 separately for with-twin / no-twin subsets so the numbers are honest.

Part 4 — Methodological choices that anchored everything

Smaller items, but each one is doing real work in defending the methodology.

4.1 Pre-registration as default

Every experiment in Part 1 has a `PROTOCOL.md` in git that was committed **before** training or measurement began. Recipe v2, Intervention 2, hubness post-processing, threshold calibration, and Recipe v3 all locked their hypothesis, primary endpoint, statistical method, decision rule, and falsification criterion before seeing any data. A reviewer can `git log -- PROTOCOL.md` and verify the commit predates the result commits.

This is what separates "we tweaked things until it worked" from "we predicted, measured, and reported honestly." Pre-registration is also what makes the *negative* results (Intervention 2, hubness) publishable as evidence — they aren't post-hoc rationalizations.

4.2 Cross-corpus tuning to avoid α-cherry-picking (Intervention 2)

Intervention 2's α-sweep could have been a hyperparameter-search-on-the-test-set in disguise. The

protocol explicitly required:

- Tune α on Hindustani only → evaluate on Carnatic at the chosen α
- Tune α on Carnatic only → evaluate on Hindustani at the chosen α
- Report **both** held-out numbers, plus the pooled-corpus number at the literature-pinned $\alpha=0.10$

This is the standard out-of-sample protocol from speaker / cross-lingual research. It prevents "find the best α on the very thing you're evaluating" — which would inflate the apparent gain.

4.3 Multiple controls per experiment

Negative experiments need controls or you can't tell *why* they failed. Intervention 2 carried three:

Control	Falsifies
$\alpha = 0$ sanity	"Maybe the rescore path has a bug"
Shuffled-centroid	"Maybe any centroid helps, regardless of artist label"
Isotropic-centroid	"Maybe the gain is the generic anti-hub effect"

The isotropic-centroid control was the one that delivered the decisive finding — matching V1 in 4/4 main cells told us the mechanism wasn't artist-specific. Without it, Intervention 2 would have been "didn't pass significance" — uninformative. With it, the result is "mechanism falsified" — actually useful for the field.

4.4 Honest disclosure in Limitations

Six MAJOR disclosures from the audit (Part 2) became the project's published Limitations section: alignment-error quantization artefacts, main/ablation overlap, tani underpowering, match_count cross-system incompatibility, latency timer asymmetry, HR@K>1 K-asymmetry. None of these change the headline HR@1, but they shape what a reader can and can't claim from the benchmark.

Disclosure is part of the deliverable. Hiding it would have been easier and shorter; including it is what makes the benchmark defensible.

Closing — how this becomes Recipe v3

Reading Parts 1-4 in order, the path is mechanical:

1. **Recipe v2** showed the recipe works directionally (HR@1 0.983 → 0.991) but single-seed × 10 ep is at the statistical-power floor → need more seeds and more epochs.
2. **Intervention 2** ruled out inference-time per-artist correction (mechanism falsified by isotropic control) → must change the encoder, not the inference path.
3. **Hubness post-processing** ruled out generic anti-hubness rescoring (zero delta on all 8 cells) → same conclusion, independent route.
4. **The audit** cleaned the methodology (twin flag, Dejavu off-by-one, score.py denominator, NAFP argsort stability) and surfaced six honest limitations → the headline numbers are now defensible at the methodology layer.

5. **Engineering gotchas** (TF_USE_LEGACY_KERAS, Kaggle phone-verify, HF Spaces pinning) are permanent memory notes → won't bite the next training run.

So when Recipe v3 happened — **3 seeds × 30 epochs × BSZ 320 × the two NMFP fixes** — every prior experiment had narrowed the search space, justified the compute, validated the statistical apparatus, and de-risked the deployment. The headline (pooled McNemar $p = 3.18 \times 10^{-6}$, Bonferroni-significant on main_1s and ablation_1s) is the easy part to report. Everything in this document is the work that made the headline possible.

The honest framing for a viva:

"Recipe v3 isn't the only thing we tried. We tried two inference-time fixes that both failed under pre-registered falsification criteria, ran a single-seed pilot that didn't clear significance, audited the benchmark with four parallel reviewers and caught four real bugs, and documented every engineering gotcha as a permanent memory note. The result we report is what survived all of that."

Appendix — File map

Every claim in this document maps to a file. Re-runnable, auditable.

Section	File(s)
Recipe v2	data/results/nafp/recipe_v2_10ep/PROTOCOL.md, recipe_v2_10ep/main_1s/scores.json
Intervention 2	data/results/nafp/intervention2/PROTOCOL.md, intervention2/analysis/SUMMARY.md, intervention2/analysis/results.csv
Hubness post-processing	data/results/nafp/hubness_postproc/main_*/scores.json
Phase 1 audit	docs/post_mortem_2026-05-12.md (294 lines, every check re-runnable)
TF_USE_LEGACY_KERAS=1 discovery	Memory note feedback_nafp_mac_inference_gotcha.md; permanent fix in demo/app.py:19
Kaggle phone-verify	Memory note feedback_kaggle_phone_verify.md; mandatory matmul assert in every Kaggle kernel
HF Spaces build cascade	demo/DEPLOY.md Troubleshooting section + demo/requirements.txt pins
Pre-registration discipline	All five PROTOCOL.md files in data/results/**/PROTOCOL.md
Recipe v3 (the headline that all of this built up to)	data/results/nafp/recipe_v3_30ep/PROTOCOL.md, RESULTS.md, pooled_mcnemar.csv

End of engineering log.

If you want the deep math behind any specific result — open docs/AFP_Teaching_Guide.pdf. For the high-level overview of the whole project — open docs/AFP_Overview_Guide.pdf. This document is the *engineering* slice: the things that didn't work, the bugs we caught, and the discipline that made the recipe v3

result publishable